

Writing Effective Test Cases

Structure • Coverage • Clarity • Maintainability

Brian Scott McCarthy | Senior QA Lead | QA Best Practice Series — BP-02

1. What Makes a Great Test Case

A great test case has **one clear goal**, can be executed by anyone on the team with zero clarification, produces a **pass or fail decision** — never a 'maybe' — and is **maintainable** as the application evolves.

Characteristic	Good	Bad
Single responsibility	Verify that a valid credit card completes checkout	Test the entire checkout flow
Atomic steps	Click the 'Add to Cart' button	Do the cart stuff
Measurable expected result	HTTP 200; cart count increments by 1; toast shows 'Added to Cart'	It works correctly
Clear preconditions	User is logged in; Product SKU-001 is in stock	App is running
Data specified	Enter email: testuser@qa.com	Enter a valid email
Independent	Does not depend on TC-0020 having passed	Assumes TC-0020 already ran

2. Anatomy of a Test Case

Field	Description	Example
ID	Unique identifier — never reuse	TC-0042
Title	[Action]-[Object]-[Condition]	PlaceOrder-ValidCard-AuthUser
Module / Suite	Feature area for organisation	Checkout — Payment Processing
Priority	P1 (critical path) to P4 (cosmetic/edge)	P1
Test Type	Functional / Negative / Regression / Smoke / API	Functional
Preconditions	Exact state required before step 1	User logged in; cart contains 1 item SKU-001; Stripe test mode active
Test Data	All inputs specified — no ambiguity	Card: 4242 4242 4242 4242 Exp: 12/26 CVV: 123
Steps	Numbered; one action per step	1. Click 'Proceed to Checkout'. 2. Select saved address...
Expected Result	Observable, specific outcome per step	Step 3: 201 response; orderId returned; confirmation email sent within 2 min
Linked User Story	ADO traceability	US-4821 — Checkout with Credit Card
Automation Status	Manual / Automated / Automation Candidate	Automated (Playwright TC-0042)

3. Writing Good Test Steps

Each step must contain exactly ONE action and ONE expected result. Never bundle two actions into a single step.

Principle	Bad Step	Good Step
One action per step	Fill in the form and click Submit	Step 1: Enter 'John' in the First Name field. Step 2: Enter 'testuser@qa.com' in the Email field. Step 3: Click 'Submit'.
Observable result	The page works	HTTP 200 response; page redirects to /dashboard; 'Welcome, John' message visible.
Exact data	Enter a password	Enter 'SecureP@ss1!' in the Password field.
No assumed knowledge	Log in	Navigate to /login. Enter email 'testuser@qa.com' and password 'SecureP@ss1!'. Click 'Sign In'.
Validate — don't just act	Click the button	Click 'Place Order'. Verify: order confirmation page displays Order ID; confirmation email received within 2 min.

4. Test Coverage Categories

Every feature should be tested across these dimensions — not just the happy path:

Coverage Type	Description	Example for Login
Positive / Happy Path	Valid inputs; expected success flow	Valid email + password → login succeeds, redirected to dashboard
Negative	Invalid inputs; expected rejection	Wrong password → error message; correct credentials not exposed
Boundary Values	At, below, and above data limits	Password max 64 chars — test 64, 65, 63
Equivalence Classes	One representative from each valid/invalid group	Valid email formats; invalid formats (no @, no TLD, spaces)
Error Handling	System responds gracefully to failures	Network timeout on login → friendly error, not stack trace
Security	Auth, session, and data protection scenarios	Session cookie flags; brute-force lockout; SQL injection in email field
Edge Cases	Unusual but valid scenarios	Login with email containing + alias; very long username
State Transitions	Behaviour changes based on prior state	Locked account → correct password still blocked; unlock → login works

5. Positive vs. Negative vs. Edge Case — Quick Reference

Scenario Type	When to Write It	Priority
Happy Path (Positive)	Always — for every user-facing feature	P1 — highest
Negative — invalid input	Always — for every input field or API parameter	P1-P2
Negative — unauthorised access	Always — for any authenticated endpoint or page	P1
Boundary value	When field has a min/max constraint	P2

Scenario Type	When to Write It	Priority
Error recovery	When the feature has network or async operations	P2
SQL injection / XSS probe	For all text input fields	P2
Edge case — unusual valid data	When real users could plausibly enter unusual values	P3
Cosmetic / display edge case	Extreme text lengths, special characters in display fields	P3-P4

6. Data-Driven Test Cases

When the same test logic applies to multiple data sets, write ONE test case with parameters rather than duplicating the test N times.

In ADO: Use the 'Parameters' tab on a test case to define a data table. Steps reference parameters using @paramName syntax. ADO creates one test iteration per data row automatically.

@email	@password	@expected_result
testuser@qa.com	SecureP@ss1!	Login succeeds — redirect to /dashboard
TESTUSER@QA.COM	SecureP@ss1!	Login succeeds (case-insensitive email)
testuser@qa.com	wrongpass	401 error — 'Invalid email or password'
notanemail	SecureP@ss1!	Validation error — 'Please enter a valid email'
		Validation error — required field messages shown

7. Automation Candidates — Decision Guide

Automate?	Criteria
YES — High priority	Executed every sprint (regression); P1/P2 critical path; stable, non-volatile UI; data-driven scenarios
YES — Medium priority	Cross-browser validation; API contract tests; DB data integrity checks post-transaction
MAYBE — Review	Complex multi-step workflow; depends on third-party system; requires visual assertion
NO — Keep manual	Exploratory / usability testing; one-time validation; highly volatile UI under active development
NO — Never automate	Tests requiring human judgment on visual design quality or content tone

8. Test Case Review Checklist

Before submitting a test case for approval, verify every item:

- Title follows TC-[#####]-[Action]-[Context] naming convention

- Linked to the correct Azure DevOps User Story
- Priority and Test Type fields populated
- Preconditions are complete and unambiguous
- All test data is specified — no 'enter a valid X'
- Each step has exactly one action
- Each step has a specific, observable expected result
- Negative and boundary scenarios are covered — not just the happy path
- Automation status is tagged (Manual / Automated / Candidate)
- Test case is independent — can run in any order